

Tabla de Contenido

Introducción.....	3
Componentes del Software.....	4
AWS.....	5
Seguridad de Acceso.....	5
Registro de Imagenes	6
Orquestacion.....	7
Load Balancer (Balanceador de carga).....	8
Automatizacion pipeline github.....	10
El Problema: Error de TIMEOUT.....	11
La Metodología de Diagnóstico (Troubleshooting).....	11
La Solución.....	12
El Análisis de los Logs en Tiempo Real.....	14
Conclusiones.....	16
Referencias Bibliográficas.....	17

Introducción

El despliegue de aplicaciones modernas exige no solo que el código sea eficiente, sino que la infraestructura que lo soporta sea capaz de actualizarse sin interrumpir la experiencia del usuario final. En entornos de producción reales, detener un servidor para subir una nueva actualización representa pérdidas económicas y una mala experiencia de usuario.

El objetivo principal de este informe es documentar el diseño, la implementación y la validación de una arquitectura de software en la nube utilizando Amazon Web Services (AWS) y GitHub Actions. A través de este entorno, se busca demostrar un flujo de Integración y Despliegue Continuo (CI/CD) completamente automatizado, logrando una actualización de versión con cero tiempo de inactividad (Zero-Downtime) sobre una API REST de alto rendimiento desarrollada en el lenguaje de programación Rust.

Componentes del Software

Antes de profundizar en la infraestructura montada dentro de AWS, es crucial delimitar las dos piezas de software que configuran la base de toda la actividad de este proyecto. El experimento no se basa en teoría abstracta, sino en la interacción directa de dos componentes ligeros diseñados para medir la disponibilidad de la red:

- **El Servidor (API REST en Rust):** Se trata de un servicio web desarrollado en Rust, elegido por su alto rendimiento, gestión eficiente de memoria y velocidad extrema de inicio. La tarea fundamental de esta API es responder de forma inmediata a peticiones HTTP con el método GET, entregando una respuesta en formato JSON con la siguiente estructura:

```
{  
  
    "mensaje": "ok",  
  
    "version": "V1.0",  
  
    "hora": "27-06-2026 23:51:06"  
  
}
```

Este JSON actúa como nuestra métrica principal; expone la versión del código activa en el contenedor (V1.0, V2.0 o Vx) y la hora exacta del servidor en cada milisegundo de respuesta.

El Cliente (Script de Monitoreo en Bash): Para poner en práctica y validar de verdad el despliegue con disponibilidad continua, se desarrolló un script automatizado en Bash. Se optó por Bash debido a su ligereza, facilidad y portabilidad, ya que permite una ejecución limpia directamente desde la terminal del sistema operativo sin necesidad de instalar entornos de ejecución pesados ni dependencias externas.

Este script ejecuta peticiones HTTP continuas (haciendo uso del comando nativo curl) en un bucle cerrado con un intervalo de entre 1 y 2 segundos. Su trabajo crítico es imprimir la respuesta en la terminal en tiempo real y, de forma simultánea, almacenar cada línea devuelta junto con un encabezado de marca de tiempo (timestamp) local dentro de un archivo de texto plano. Esto genera una especie de logs que nos permitirá evaluar el comportamiento del tráfico durante la transición de versiones.

Ya que el punto central de este proyecto no es el código en sí, no se detallará mucho más acerca de este a lo largo del informe por lo que si se desea ver la estructura o código puede encontrar más información en el repositorio donde también puede encontrar los archivos dockerfile y docker-compose para construir y levantar un contenedor con el api.

<https://github.com/Variouslamp/Bluegreen-api#>

AWS

Seguridad de Acceso

Tras el inicio de sesión en una cuenta raíz (Root) en AWS, el primer paso fundamental bajo las buenas prácticas de la arquitectura de la nube no es crear servidores directamente, sino asegurar las entradas del sistema. Para gestionar el control de accesos, se utilizó el servicio AWS IAM (Identity and Access Management).

Para interactuar con la infraestructura de AWS desde plataformas externas, como la terminal de la computadora local o los corredores virtuales (runners) de GitHub Actions, representa una pésima práctica de seguridad utilizar las credenciales de la cuenta principal. El

comportamiento de los usuarios en IAM es idéntico a cómo funcionan los usuarios dentro de un sistema operativo Linux: actúan como puntos de entrada controlados al sistema. De la misma manera en que a un usuario común en Linux no se le otorgan permisos de administrador para todo, en AWS se crearon usuarios específicos con permisos estrictamente acotados al principio de menor privilegio. Si el pipeline automatizado solo necesita compilar y subir imágenes de contenedores, no tiene por qué poseer permisos para ver la facturación o eliminar bases de datos.

A este usuario técnico dedicado se le generaron Llaves de Acceso, compuestas por un par de llaves (Access Key ID y un Secret Access Key). Estas llaves operan como un usuario y contraseña digitales de alta seguridad para APIs. Posteriormente, fueron inyectadas de manera segura dentro de los secretos del repositorio de GitHub. De esta manera, el pipeline de automatización puede firmar digitalmente cada petición que le hace a Amazon de forma segura y totalmente transparente, sin exponer credenciales en el código fuente.

Registro de Imagenes

Una vez resuelta la seguridad de acceso, la arquitectura requería un componente centralizado y seguro para almacenar las imagenes de las aplicaciones. Para cumplir con esta función, se desplegó el servicio AWS ECR (Elastic Container Registry).

La manera más sencilla de entender el propósito de AWS ECR es visualizarlo como un "Docker Hub Privado" que vive dentro de AWS. Su único objetivo en este diseño es servir como el almacén oficial de artefactos. Cuando el pipeline automatizado de GitHub Actions termina de compilar el código fuente de nuestra API en Rust y construye la imagen Docker, esta no se envía directamente a los servidores de ejecución, sino que se sube (push) a este repositorio en ECR.

Cada imagen guardada en ECR se indexa de forma automática con etiquetas de versión específicas. Esto nos garantiza un control absoluto sobre el historial de despliegues y permite que el orquestador de contenedores de AWS descargue las imágenes de manera interna a máxima velocidad, protegiendo la integridad del código del servidor frente al internet público.

Orquestacion

Con las imágenes de Docker guardadas de forma segura en ECR, el siguiente paso consiste en crear el entorno donde esas imágenes se transformarían en contenedores en ejecución. Para esto se utilizó AWS ECS (Elastic Container Service) bajo la modalidad de lanzamiento AWS Fargate.

La ventaja crítica de utilizar AWS Fargate en esta arquitectura es su naturaleza Serverless (sin servidor) para contenedores. En los esquemas tradicionales en la nube, se requiere alquilar, configurar y configurar sistemas operativos en servidores virtuales (como instancias EC2 o VPS), lo cual consume tiempo de administración y mantenimiento. Fargate elimina esta capa de complejidad por completo: nosotros simplemente le indicamos cuánta memoria RAM y cuánta CPU necesita nuestra API de Rust, y Amazon se encarga de aprovisionar y aislar la infraestructura subyacente de forma automática.

Para usar este entorno, se configuraron dos abstracciones clave dentro de ECS:

1. **La Definición de la Tarea (Task Definition):** Funciona como el plano de construcción o el plano arquitectónico del contenedor. Aquí se especificó de qué repositorio de ECR debe descargarse la imagen, los límites de hardware (CPU/RAM) y, de manera crucial, que la aplicación de Rust expone internamente el puerto 8080 para escuchar el tráfico.

Se puede encontrar el archivo de definicion dentro del repositorio que se dejo mas arriba.

2. **El Servicio (Service):** Es el supervisor encargado de mantener el estado deseado de la aplicación. Su función primordial es garantizar que siempre haya al menos una réplica del contenedor corriendo de forma saludable. Si un contenedor llegara a fallar de forma inesperada, el Servicio lo detecta de inmediato, lo destruye y levanta uno nuevo basados en la Task Definition para asegurar la continuidad del negocio.

Load Balancer (Balanceador de carga)

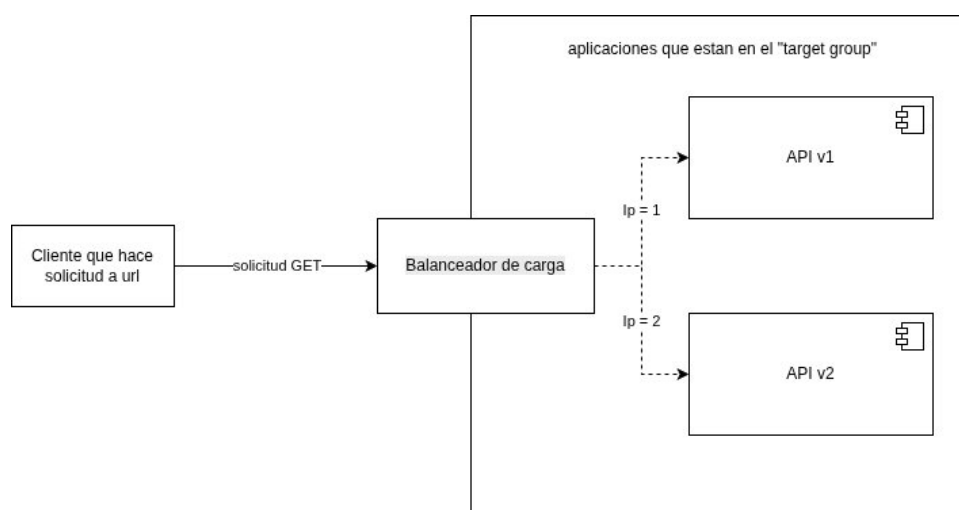
Aunque el contenedor de Rust se encuentre corriendo de manera óptima dentro de AWS ECS Fargate, este vive aislado dentro de la red interna de Amazon y su IP cambia constantemente cada vez que se reinicia o actualiza. Para exponer la aplicación hacia el exterior de forma segura y unificada, se implementó un Application Load Balancer apoyado por un Target Group (Grupo Objetivo).

Este componente actúa como el "guardián y distribuidor" del tráfico mediante dos mecanismos perfectamente acoplados:

1. **El Listener (Oyente en el Puerto 80):** El Load Balancer da la cara a internet a través de una URL. Este se configuró con un Listener que escucha en el puerto estándar 80 (HTTP). Esto significa que cualquier cliente externo (como nuestro script de Bash, la herramienta atac o postman) puede enviarle peticiones de forma limpia sin necesidad de especificar puertos extraños en la URL.

2. **El Target Group (Puerto 8080 y Monitoreo):** El Load Balancer no procesa la lógica del negocio; su trabajo es redirigir el tráfico hacia el Target Group. Este grupo actúa como un grupo dinámico que sabe exactamente qué IPs privadas tienen los contenedores de Rust activos en cada segundo. El Target Group toma la petición que entró por el puerto 80 y la inyecta directamente en el puerto 8080 del contenedor.

Adicionalmente, el Target Group se encarga del Health Check (Control de Salud). Cada cierto intervalo de tiempo, le hace una petición automática a la API. Si el contenedor responde con un código HTTP exitoso (200 OK), el balanceador lo mantiene en la lista de servidores activos. Si el contenedor no responde, lo marca como insalubre y desvía el tráfico de inmediato hacia una réplica sana, garantizando que el usuario final nunca reciba una conexión rota.



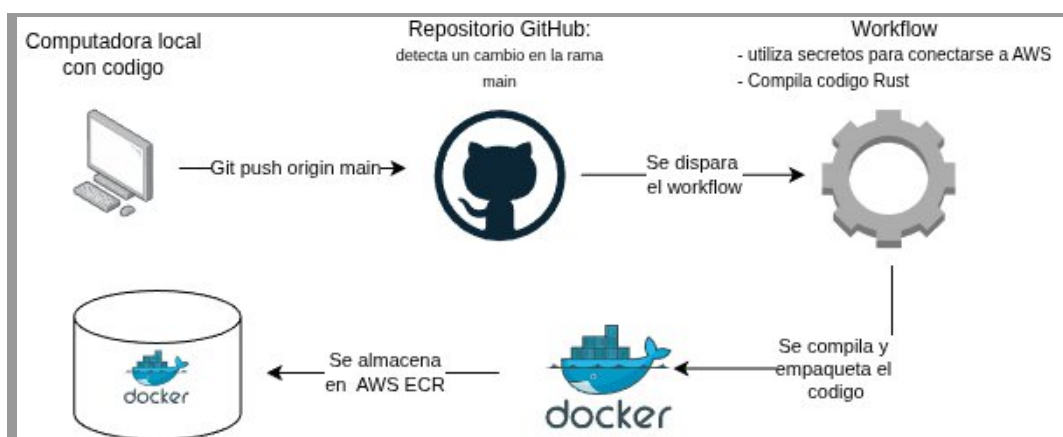
El balanceador de carga hace el papel de porte o unico punto de entrada y a partir de el se reparten las respuestas a una u otra aplicacion.

Automatización pipeline github

La infraestructura montada con ECS y el Load Balancer garantiza que la aplicación sea accesible, pero el verdadero poder de esta arquitectura radica en la automatización del despliegue. Para lograr un flujo ágil e industrial, se configuró un pipeline de GitHub Actions que reacciona de forma inmediata ante cualquier cambio en el código fuente.

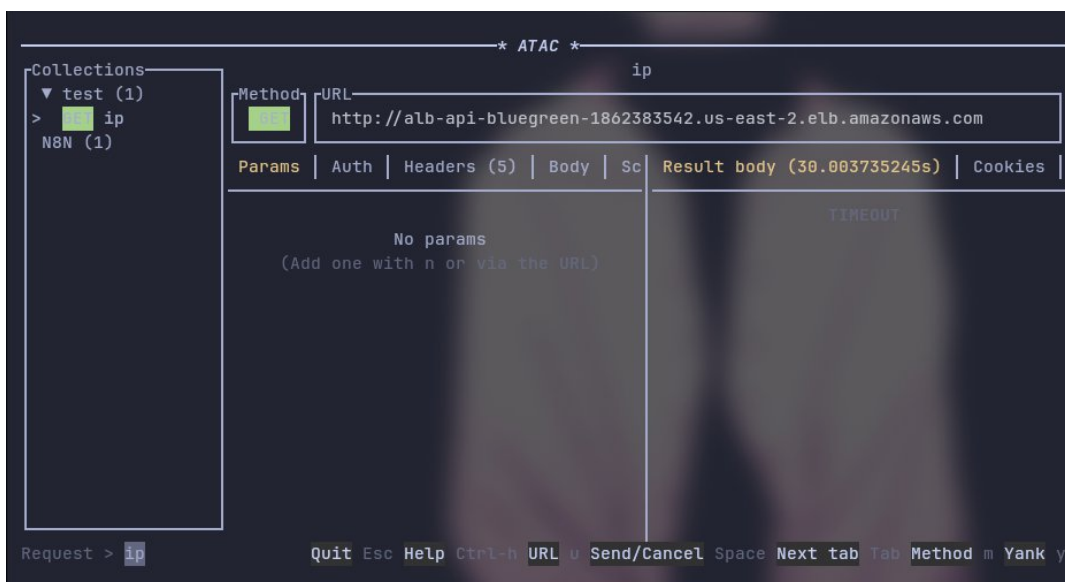
Cuando el desarrollador ejecuta un comando `git push origin main`, se dispara un flujo de trabajo automatizado (Workflow) compuesto por las siguientes etapas:

1. **Autenticación en AWS:** Utilizando las credenciales seguras de IAM almacenadas en los Secrets del repositorio, el pipeline inicia sesión en el panel de AWS.
2. **Construcción y Compilación en Docker:** El corredor virtual levanta la API en Rust, compila el código binario de alto rendimiento y empaqueta el resultado en una imagen Docker optimizada.
3. **Push a AWS ECR:** La imagen recién empaquetada se sube directamente al repositorio privado en ECR con una nueva etiqueta de versión.
4. **Actualización del Servicio en ECS:** GitHub Actions le notifica a AWS ECS que hay una nueva Task Definition disponible apuntando a la nueva imagen. Esto dispara de forma nativa la estrategia de actualización en caliente por parte de Amazon.



El Problema: Error de TIMEOUT

Una vez que el contenedor de Rust se encontraba corriendo de forma saludable dentro de AWS ECS y el Application Load Balancer estaba encendido, se intentaron realizar las primeras peticiones de prueba utilizando el cliente de API REST y la aplicación ATAC. El resultado fue un bloqueo total en la conexión: la terminal quedaba colgada indefinidamente hasta arrojar un error de TIMEOUT (Tiempo de espera agotado).



ATAC es una herramienta tipo postman que permite hacer llamadas http con distintos métodos (GET, POST, PUT, etc)

La Metodología de Diagnóstico (Troubleshooting)

Un error de *Timeout* en redes de AWS es el síntoma inequívoco de que los paquetes de datos están siendo descartados silenciosamente por un firewall; el cliente envía la petición, pero jamás recibe una respuesta de confirmación, ni siquiera un rechazo explícito.

Para rastrear la falla, se aisló el problema capa por capa:

1. Se verificaron los logs internos de la aplicación en Rust dentro de ECS CloudWatch, confirmando que el servidor estaba encendido y escuchando el puerto 8080.
2. Se revisaron los *Health Checks* del Target Group, detectando que el balanceador de carga lograba establecer comunicación con los contenedores.
3. Se concluyó que el bloqueo se encontraba en la capa de seguridad externa, la que permite recibir información desde el exterior (internet) y que esta sea procesada por el balanceador de carga

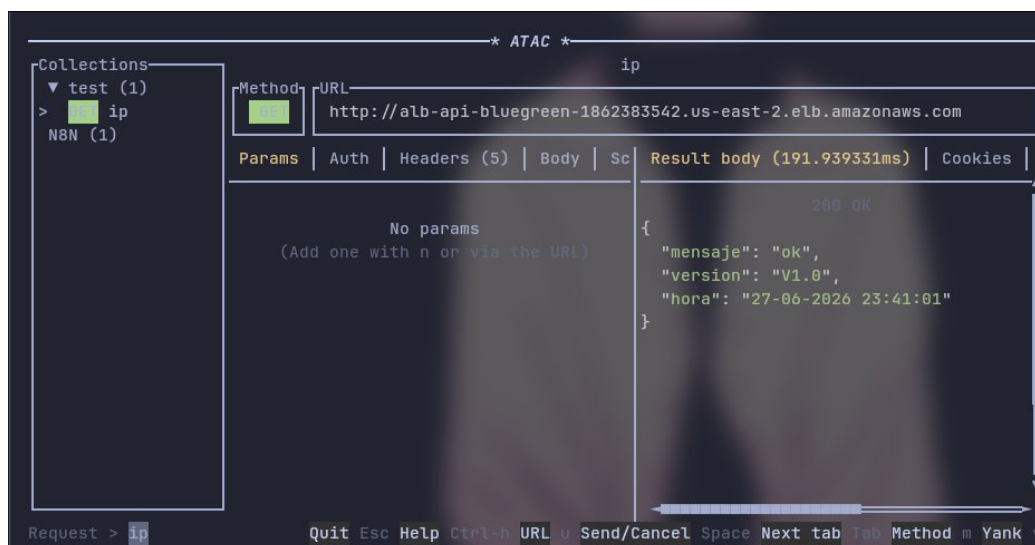
La Solución

El error radicaba en que el Security Group (Grupo de reglas de seguridad, que evitan el ingreso de información a las aplicaciones) asociado al balanceador de carga ya venía cerrado por defecto para conexiones externas. Al actuar como un firewall, estaba bloqueando todo el tráfico entrante.

La solución consistió en editar las reglas de dicho Grupo de Seguridad, aplicando las siguientes directivas:

- **Tipo de Tráfico:** HTTP
- **Puerto:** 80 (El puerto público del balanceador)
- **Origen:** 0.0.0.0/0 (Cualquier dirección IP en internet)

Inmediatamente después de aplicar este cambio en caliente y guardar las reglas en la consola de AWS, la herramienta ATAC rompió el bloqueo, recibiendo de forma instantánea el código HTTP 200 ok, junto con el JSON estructurado de la API. Con este cableado de red corregido, la autopista quedó completamente libre para iniciar el experimento de disponibilidad continua.



Disponibilidad Continua

Con la infraestructura correctamente cableada y el canal de comunicación libre de bloqueos, se ejecutó la prueba reina del laboratorio: validar un despliegue automatizado sin caída del servicio (*Zero-Downtime Deployment*). Para ello, se activó el script de monitoreo en Bash en la terminal de control local, apuntando directamente a la dirección DNS pública del Application Load Balancer.

Mientras el cliente realizaba peticiones continuas segundo a segundo registrando la versión activa en ese instante (V1.0), se realizó una modificación en el código de la API en Rust para actualizar su salida a la versión (V2.0) Posteriormente, se ejecutó un comando ***git push origin main*** con el commit "***fix(Bluegreen-api): Launch new version v2.0***" (que se puede encontrar en el repositorio), disparando el pipeline de GitHub Actions.

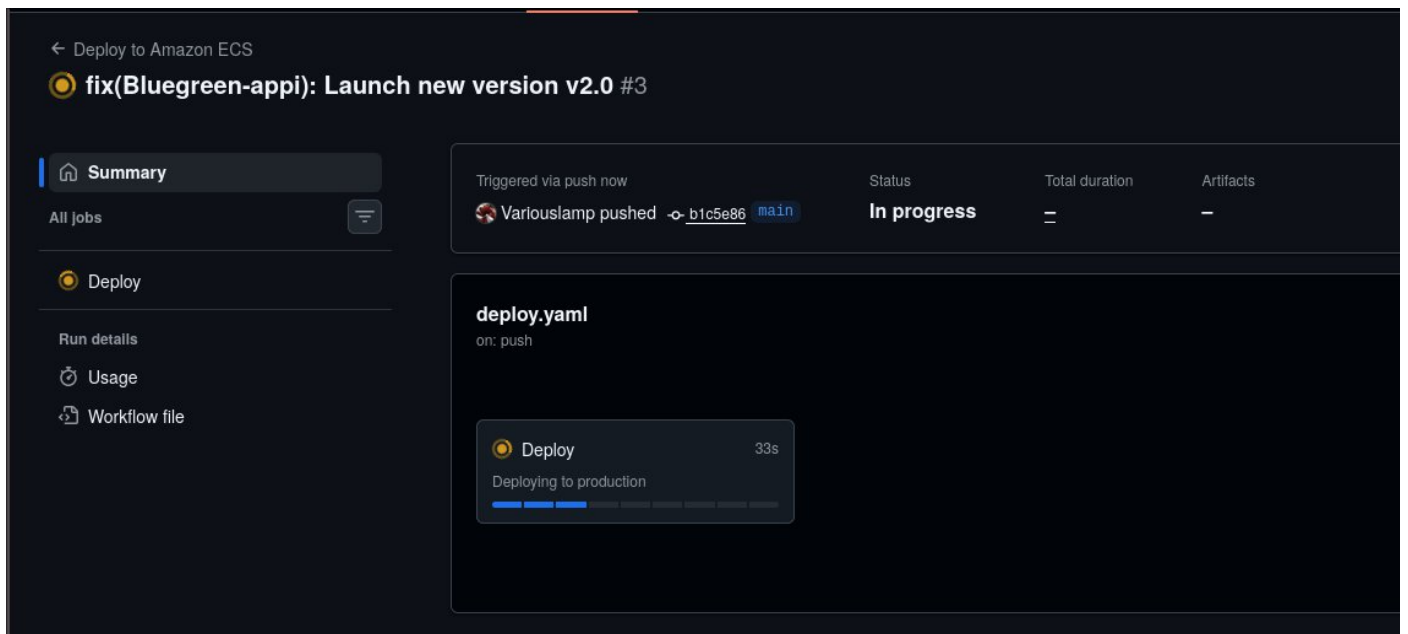


Imagen que muestra el proceso de automatizacion inicializado por el commit en la rama main.

El Análisis de los Logs en Tiempo Real

El script de Bash generó un archivo de texto plano que capturó de manera milimétrica el comportamiento del Application Load Balancer durante el despliegue. A continuación, se presenta y analiza el fragmento crítico de la bitácora forense obtenida:

Se plasmaran una parte muy pequeña de los logs del cliente mientras se hacia el cambio de un servicio a otro, este proceso cuenta con 3 etapas principales las cuales son:

1. Etapa en la que aun no se habia compilado ningun codigo ni nada y la unica API en la nube que nos respondia era la V1.0
2. Etapa en que el load balancer empieza a intercalar las llamadas entre las dos apis hasta que estipule que la api V2.0 sea lo suficientemente estable para colocarla como unica y principal

3. Etapa en la que la API V2.0 toma control total y solo ella responde a los llamados de api

A continuacion una seccion de los logs:

(Inicio de la etapa 1, momento del la inicializacion del cliente)

```
{"mensaje":"ok","version":"V1.0","hora":"27-06-2026 23:49:58"}
```

(Inicio de etapa 2, primera aparicion de la V2.0)

```
{"mensaje":"ok","version":"V2.0","hora":"27-06-2026 23:51:58"}
{"mensaje":"ok","version":"V1.0","hora":"27-06-2026 23:51:59"}
{"mensaje":"ok","version":"V1.0","hora":"27-06-2026 23:52:01"}
{"mensaje":"ok","version":"V2.0","hora":"27-06-2026 23:52:02"}
{"mensaje":"ok","version":"V1.0","hora":"27-06-2026 23:52:03"}
{"mensaje":"ok","version":"V2.0","hora":"27-06-2026 23:52:04"}
{"mensaje":"ok","version":"V1.0","hora":"27-06-2026 23:52:05"}
{"mensaje":"ok","version":"V2.0","hora":"27-06-2026 23:52:07"}
[continua con el intercambio ...]
```

(Inicio de la etapa 3, ultimo uso de api V1.0)

```
{"mensaje":"ok","version":"V1.0","hora":"27-06-2026 23:52:54"}
```

(De ahi en adelante la V2.0 es la que siempre responde a los rquest)

```
04:44 ~/projects-dev/Bluegreen-api/check-script main
) cat salida.txt | grep v
{"mensaje":"ok","version":"V1.0","hora":"27-06-2026 23:49:58"}
{"mensaje":"ok","version":"V1.0","hora":"27-06-2026 23:49:59"}
{"mensaje":"ok","version":"V1.0","hora":"27-06-2026 23:50:01"}
{"mensaje":"ok","version":"V1.0","hora":"27-06-2026 23:50:02"}
{"mensaje":"ok","version":"V1.0","hora":"27-06-2026 23:50:03"}
{"mensaje":"ok","version":"V1.0","hora":"27-06-2026 23:50:04"}
{"mensaje":"ok","version":"V1.0","hora":"27-06-2026 23:50:05"}
{"mensaje":"ok","version":"V1.0","hora":"27-06-2026 23:50:06"}
{"mensaje":"ok","version":"V1.0","hora":"27-06-2026 23:50:08"}
{"mensaje":"ok","version":"V1.0","hora":"27-06-2026 23:50:09"}
{"mensaje":"ok","version":"V1.0","hora":"27-06-2026 23:50:10"}
{"mensaje":"ok","version":"V1.0","hora":"27-06-2026 23:50:11"}
{"mensaje":"ok","version":"V1.0","hora":"27-06-2026 23:50:12"}
{"mensaje":"ok","version":"V1.0","hora":"27-06-2026 23:50:14"}
{"mensaje":"ok","version":"V1.0","hora":"27-06-2026 23:50:15"}
{"mensaje":"ok","version":"V1.0","hora":"27-06-2026 23:50:16"}
{"mensaje":"ok","version":"V1.0","hora":"27-06-2026 23:50:17"}
{"mensaje":"ok","version":"V1.0","hora":"27-06-2026 23:50:18"}
{"mensaje":"ok","version":"V1.0","hora":"27-06-2026 23:50:20"}
{"mensaje":"ok","version":"V1.0","hora":"27-06-2026 23:50:21"}
{"mensaje":"ok","version":"V1.0","hora":"27-06-2026 23:50:22"}
{"mensaje":"ok","version":"V1.0","hora":"27-06-2026 23:50:23"}
{"mensaje":"ok","version":"V1.0","hora":"27-06-2026 23:50:24"}
{"mensaje":"ok","version":"V1.0","hora":"27-06-2026 23:50:25"}
{"mensaje":"ok","version":"V1.0","hora":"27-06-2026 23:50:27"}
{"mensaje":"ok","version":"V1.0","hora":"27-06-2026 23:50:28"}
{"mensaje":"ok","version":"V1.0","hora":"27-06-2026 23:50:29"}
{"mensaje":"ok","version":"V1.0","hora":"27-06-2026 23:50:30"}
{"mensaje":"ok","version":"V1.0","hora":"27-06-2026 23:50:31"}
```

Evidencia de pequeña parte del contenido del archivo “salida.txt” donde se almacena toda la información de los logs de las request que hace el cliente a una url.

Conclusiones

El experimento demostró de manera contundente que es totalmente viable actualizar un servicio crítico en caliente sin perjudicar la experiencia del usuario. La bitácora forense de logs fue la prueba reina: un flujo constante de peticiones HTTP absorbió una transición completa de versión de software sin registrar una sola caída, bloqueo o pérdida de paquetes de red.

La elección de tecnologías demostró un acoplamiento óptimo. La naturaleza ligera e instantánea de los binarios compilados en Rust, combinada con la elasticidad *Serverless* de AWS Fargate, permitió que el periodo de coexistencia e intercalado de tráfico durante el *Rolling Update* durara apenas 58 segundos. Esto reduce drásticamente las ventanas de exposición durante despliegues en comparación con tecnologías tradicionales más pesadas.

El bache técnico enfrentado con el *Security Group* aportó el valor más enriquecedor del laboratorio. Evidenció que la ingeniería de nube no solo consiste en automatizar flujos exitosos, sino en poseer la capacidad analítica de rastrear paquetes mediante herramientas de diagnóstico (como *atac*), aislar componentes y comprender las políticas de seguridad por defecto que gobiernan los entornos de producción.

La automatización lograda mediante GitHub Actions eliminó por completo el error humano en el proceso de liberación. Un simple comando `git push` local desencadenó una secuencia compleja de compilación, empaquetado, almacenamiento seguro en ECR y orquestación en ECS de forma totalmente transparente, demostrando cómo una infraestructura bien diseñada permite a los equipos de desarrollo enfocarse en el código mientras la nube se encarga de la estabilidad global.